



# SWE 4603

Software Testing and Quality Assurance

Week 10

Prepared By Maliha Noushin Raida, Lecturer, CSE  
Islamic University of Technology

## Lesson Outcome

- Testing Metrics
- Progress metrics
- Quality metrics
- Cost metrics
- Size metrics
- Testing effort estimation model

## Week on:

- Chapter 11: Testing Metrics for Monitoring and Controlling the Testing Process

# Testing Metrics

# Testing Metrics

- ❖ Software testing metrics are a way to measure and monitor your test activities.
- ❖ More importantly, they give insights into your team's test progress, productivity, and the quality of the system under test.
- ❖ A metric usually conveys a result or a prediction based off the combination of data.

**Result Metrics:** metrics that are mostly an absolute measure of an activity/process completed.

Example: Time taken to run a set of test cases in a suite

**Predictive Metrics:** metrics that are **derivatives and act as early warning signs of an unfavorable result.**

Example: Defects created vs. Resolved chart shows the rate of defect fixing. This grabs the team's attention if this rate is slower than the rate desired.

# Why Testing Metrics

- ❖ The aim of collecting test metrics is to use the data for improving the test process.
- ❖ This includes finding tangible answers to the following questions:
  - How long will it take to test?
  - How bad are the bugs?
  - How many bugs found were fixed? reopened? closed? deferred?
  - How many bugs did the test team not find?
  - How much of the software was tested?
  - How good were the tests? Are we using low-value test cases?
  - What is the cost of testing?
  - Was the test effort adequate? Could we have fit more testing in this release?

Proper **measurement** answers all these questions.

# Testing Metrics

An organization needs to have a reference set of measurable attributes and corresponding metrics that can be applied at various stages during the course of execution of an organization-wide testing strategy.

Measurable attributes of software testing are,

- ❖ Progress
- ❖ Cost
- ❖ Quality
- ❖ Size
- ❖ Effort

# Progress Metrics

**Tracking test progress:** The progress of testing should also be measured to keep in line with the schedule, budget, and resources.

For measuring the test progress, well-planned test plans are prepared wherein testing milestones are planned. For example: executing all planned unit tests is one example of a testing milestone.

$$\text{Passed Test Cases Percentage} = \left( \frac{\text{Number of Passed Tests}}{\text{Total number of tests executed}} \right) \times 100$$

$$\text{Failed Test Cases Percentage} = \left( \frac{\text{Number of Failed Tests}}{\text{Total number of tests executed}} \right) \times 100$$

$$\text{Blocked Test Cases Percentage} = \left( \frac{\text{Number of Blocked Tests}}{\text{Total number of tests executed}} \right) \times 100$$

$$\text{Average time for development team to repair defects} = \left( \frac{\text{Total time taken for bug fixes}}{\text{Number of bugs}} \right)$$

# Progress Metrics

## Tracking test progress

$$\text{Accepted Defects Percentage} = \left( \frac{\text{Defects Accepted as Valid by Dev Team}}{\text{Total defects reported}} \right) \times 100$$

$$\text{Fixed Defects Percentage} = \left( \frac{\text{Defects Fixed}}{\text{Defects Reported}} \right) \times 100$$

$$\text{Defects Rejected Percentage} = \left( \frac{\text{Defects Rejected as invalid by Dev Team}}{\text{Total defects reported}} \right) \times 100$$

$$\text{Defects Deferred Percentage} = \left( \frac{\text{Defects deferred for future releases}}{\text{Total defects reported}} \right) \times 100$$

$$\text{Critical Defects Percentage} = \left( \frac{\text{Critical Defects}}{\text{Total defects reported}} \right) \times 100$$



# Progress Metrics

## S-Curve in Representation:

### What is an S-Curve?

- ❖ An S-curve is a graphical representation of cumulative work effort.
- ❖ S-curves can be used to describe projects as a whole, development efforts, test efforts, as well as defect discovery rates.

### What makes it an "S" shape?

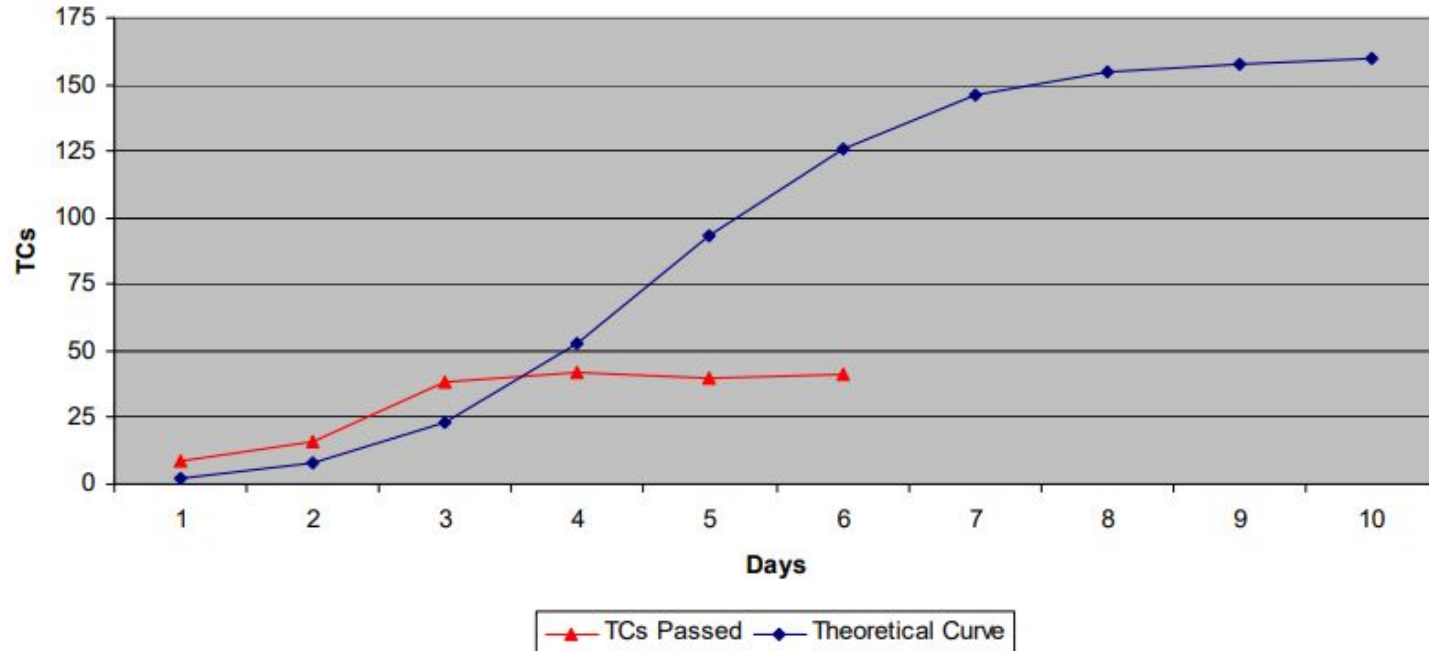
- ❖ Test efforts typically start out slowly as testers run into a few major defects
- ❖ As the initial issues are resolved, the testers are able to execute more tests covering more functionality.
- ❖ As the test effort nears its end, there are typically a few leftover issues that must be resolved, thus slowing the process down again

### How is it used?

- ❖ Measure test progress by comparing the actual test curve to a theoretical S-curve.
- ❖ Use the curve to determine if the application is stable enough to be released

## S-Curve in Representation:

Test Metrics Graph - TCs Passed



What are some potential causes associated with this S-Curve? How might you correct these issues?

# Progress Metrics

## **Defect Backlog:**

It is the number of defects that are outstanding and unresolved at one point in time with respect to the number of defects occurring. The defect backlog metric is to be measured for each release of a software product for release-to-release comparisons.

## **Staff productivity**

The basic and useful measures of a tester's productivity are:

- ❖ Time spent in test planning.
- ❖ Time spent in test case design.
- ❖ Number of test cases developed.
- ❖ Number of test cases developed per unit time.

# Cost Metrics

Infrastructure and tools contribute to the expense of testing. Testing systems do not have fathomless financial resources to spend. Thus, estimating how much you can spend and how much you indeed wrap up spending is eventful.

## **Budget variance:**

The variance between the actual and planned costs is referred to as budget variance.

## **Schedule variance:**

It is the difference between the actual time taken to complete tests and the planned time.

## **Cost per bug fix:**

It refers to the amount of effort spent on a defect per developer.

However, the cost of a bug fix is  $10 * \$60 = \$600$ ; if a developer spends 10 hours fixing a defect, their hourly rate is \$60.

# Quality Metrics

## Effectiveness of test cases

$$\text{Number of bugs per test} = \frac{\text{Total number of defects}}{\text{Total number of tests}}$$

$$\begin{array}{l} \text{Bug find rate or} \\ \text{Number of defects} \\ \text{per test hour} \end{array} = \frac{\text{Total number of defects}}{\text{Total number of test hours}}$$

$$\text{Defect-removal efficiency} = \left( \frac{\text{Bugs found in Test}}{\text{Total bugs found (bugs found in test + bugs found after shipping)}} \right) \times 100$$

## Context Based: Test Effectiveness Using Team Assessment

- ❖ You can ask your team to rate the test set for how good it is.
- ❖ Your team could also use a subjective scaling method. Out of a 100% rating (1 to 10 scale), ask your team to give a score to the test set as to how complete, up-to-date, and effective the test set is today.
- ❖ Get an average on the score to get the team's perceived average test effectiveness.
- ❖ It is important to tell your team to be unbiased and to define what a good test set means.

# Quality Metrics

## Effectiveness of test cases

Defect spoilage depends on the defect age or the phase in which it is discovered.

$$\text{Spoilage} = \frac{\text{Sum of (Number of defects} \times \text{Defect age)}}{\text{Total number of defects}}$$

Defect age = Closing date of bug – Start date when bug was opened

Generally, low values of defect spoilage mean more effective defect discovery processes.

| SDLC phase         | No. of defects | Defect age |
|--------------------|----------------|------------|
| Requirement Specs. | 34             | 2          |
| HLD                | 25             | 4          |
| LLD                | 17             | 5          |
| Coding             | 10             | 6          |

### ***Solution***

$$\text{Spoilage} = (34 \times 2 + 25 \times 4 + 17 \times 5 + 10 \times 6) / 86 = 3.64$$

# Effort Metrics

## Function points

- ❖ The function point ( FP) metric is used effectively for measuring the size of a software system.
- ❖ Function-based metrics can be used as a predictor for the overall testing effort.

Some derived metrics from FP

*Defect Density = Number of defects / Total number of FPs*

*Test Coverage = Number of test cases / Total number of FPs*

*Number of total test cases = (function points)<sup>1.2</sup>*

*Number of acceptance test cases = (function points) × 1.2*

# Effort Metrics

## Example from 11.3 Page 334

In a project, the estimated function points are 760. Calculate the total number of test cases in the system and the number of test cases in acceptance testing. Also, calculate the defect density (number of total defects is 456) and test case coverage.

## Solution

Total number of test cases =  $(760)^{1.2} = 2864$

Number of test cases for acceptance testing =  $(760) \times 1.2 = 912$

Defect density =  $456/760 = 0.6$

Test case coverage =  $2864/760 = 3.768$



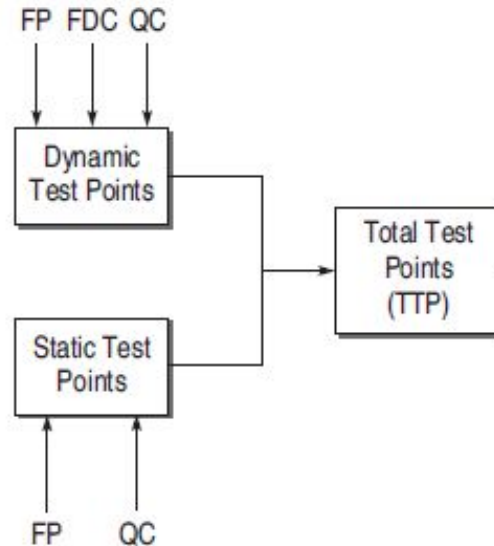
# Effort Metrics (Test Point Analysis)

- ❖ Test point analysis is a technique to measure the black-box test effort estimation.
- ❖ The technique uses function point analysis in the estimation.
- ❖ TPA calculates the **test effort** estimation in test points for highly important functions, according to the user and also for the whole system.

# Test Point Analysis

## PROCEDURE FOR CALCULATING TPA:

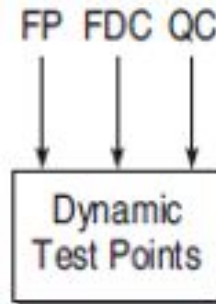
- ❖ First the dynamic test points
- ❖ Then static test points are calculated
- ❖ A dynamic test point is added to the static test point to get the **total test points (TTP)** for the system.



# Test Point Analysis

## Dynamic Test Point

- ❖ Dynamic test points are the number of test points which are based on dynamic measurable quality characteristics of functions in the system.
- ❖ To calculate a dynamic function point, we need the following:
  - **Function points** (FPs) assigned to the function.
  - **Function dependent factors** (FDC), such as complexity, interfacing, function-importance, etc.
  - Dynamic **Quality characteristics** (QC).



# Test Point Analysis

## Dynamic Test Point

The number of dynamic test points for each function in the system is calculated as

$$DTP = FP \times FDC_w \times QC_{dw}$$

where,

DTP = number of dynamic test points

FP = **function point** assigned to function.

$FDC_w$  = weight-assigned **function-dependent factors**

$QC_{dw}$  = **quality characteristic** factor, wherein weights are assigned to dynamic quality characteristics

# Test Point Analysis

## Dynamic Test Point

FDC<sub>w</sub> is calculated as

$$\text{FDC}_w = ((\text{FI}_w + \text{UIN}_w + I + C) \div 20) \times U$$

where

FI<sub>w</sub> = function importance rated by users

UIN<sub>w</sub> = weights **given to usage intensity of the function**, i.e. how frequently the function is being used

I = weights **given to the function for interfacing with other functions**, i.e. if there is a change in function, how many functions in the system will be affected

C = weights given to the complexity of function, i.e. how many conditions are in the algorithm of function

U = uniformity factor

# Test Point Analysis

## Dynamic Test Point

| Factor/<br>Rating | Function<br>Importance<br>(FI) | Function<br>usage<br>Intensity (UIN) | Interfacing<br>(I) | Complexity<br>(C) | Uniformity Factor                                                                                                                                  |
|-------------------|--------------------------------|--------------------------------------|--------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Low               | 3                              | 2                                    | 2                  | 3                 | 0.6 for the function, wherein<br>test specifications are largely<br>re-used such as in clone function<br>or dummy function. Otherwise,<br>it is 1. |
| Normal            | 6                              | 4                                    | 4                  | 6                 |                                                                                                                                                    |
| High              | 12                             | 12                                   | 8                  | 12                |                                                                                                                                                    |

# Test Point Analysis

## Dynamic Test Point

- ❖  $QC_{dw}$  is calculated based on four dynamic quality characteristics, namely **functionality, security, usability, and efficiency**.
- ❖ First, the rating for every quality characteristic is given, and then a weight for every QC is provided.
- ❖ Based on the rating and weights,  $QC_{dw}$  is calculated.

$$QC_{dw} = (QC_f * 0.75 + QC_s * 0.05 + QC_u * 0.1 + QC_e * 0.1) / 4$$

$QC_f$  = Functionality Factor

$QC_s$  = Security Factor

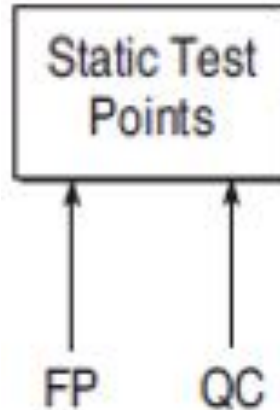
$QC_u$  = Usability Factor

$QC_e$  = Efficiency Factor

|        | Not Important | Relatively Unimportant | Medium Importance | Very Important | Extremely Important |
|--------|---------------|------------------------|-------------------|----------------|---------------------|
| Rating | 0             | 3                      | 4                 | 5              | 6                   |

# Test Point Analysis

- ❖ **Static test points** are the number of test points that are based on static quality characteristics of the system.
- ❖ It is calculated based on the following:
  - Function points (FPs) assigned to the system.
  - Quality requirements or test strategy for **static quality characteristics** (QC).





# Test Point Analysis

The number of static test points for the system is calculated as

$$\text{STP} = \text{FP} \times (\sum \text{QC}_{\text{sw}} / 500)$$

where

STP = static test point

FP = total function point assigned to the system

$\text{QC}_{\text{sw}}$  = quality characteristic factor, wherein weights are assigned to static quality characteristics

- ❖ A static quality characteristic refers to what can be tested with a checklist.
- ❖  $\text{QC}_{\text{sw}}$  is assigned the value **16** for each of **6 ISO 9126** quality characteristics, which can be tested statically using the checklist.

$$\text{Total test points (TTP)} = \text{DTP} + \text{STP}$$

# Test Point Analysis

## Example

Calculate the total test points for a module whose specifications are: **function points** = 414, ratings for all **FDC<sub>w</sub>** factors are normal, **uniformity factor** = 1, rating for all **QC<sub>dw</sub>** are 'very important,' and for **QC<sub>sw</sub>**, three static qualities are considered.

$$\text{FDC}_w = ((\text{FI}_w + \text{UIN}_w + \text{I} + \text{C}) / 20) \times \text{U} = ((6+4+4+6)/20) \times 1 = 1$$

$$\begin{aligned}\text{QC}_{dw} &= \sum (\text{rating of QC} / 4) \times \text{weight factor of QC} \\ &= (5/4 \times 0.75) + (5/4 \times 0.05) + (5/4 \times 0.10) + (5/4 \times 0.10) \\ &= 0.9375 + 0.0625 + 0.125 + 0.125 = 1.25\end{aligned}$$

$$\begin{aligned}\text{DTP} &= \text{FP} \times \text{FDC}_w \times \text{QC}_{dw} \\ &= 414 \times 1 \times 1.25 = 517.5\end{aligned}$$

$$\begin{aligned}\text{STP} &= \text{FP} \times \sum \text{QC}_{sw} / 500 \\ &= 414 \times ((16 \times 3)/500) = 39.744\end{aligned}$$

$$\text{Total test points (TTP)} = \text{DTP} + \text{STP} = 517.5 + 39.744 = 557.244$$